DAG and TopologicalSort

2026-1학기 부경대학교 알고리즘 스터디

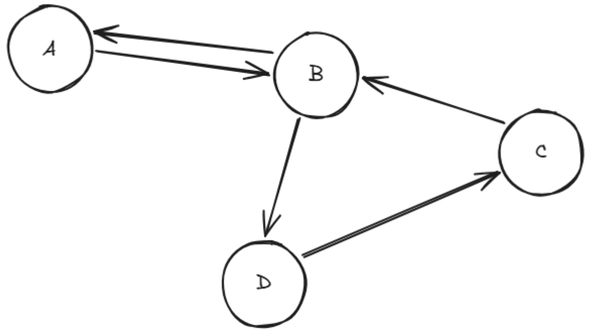
목차

- 방향 그래프(Directed Graph)와 차수(degree)
- 방향성 비순환 그래프(Directed Acyclic Graph)
- 위상 정렬(Topological Sort)
- Kahn의 알고리즘
- 동적 계획법과(DP)과 DAG

방향 그래프 심화

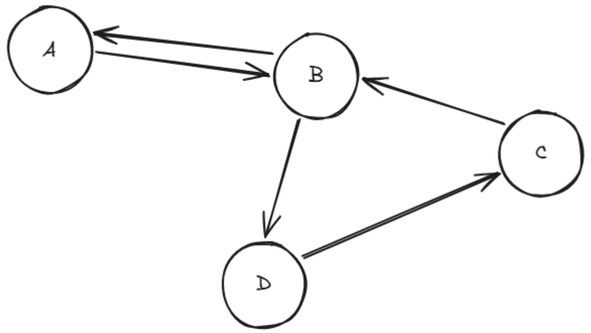
- 방향 그래프 (Directed Graph)
  - 간선에 방향이 존재하는 일방통행 그래프입니다.
  - 단순한 연결을 넘어, 정점 간의 선후 관계나 위계를 표현하는데 사용될 수 있습니다.
  - 방향 그래프에서 사이클을 이루기 위해서는 최소 2개의 정점이 필요합니다.

    ※ 무방향 그래프에서는 최소 3개의 점점이 필요
    ※ 단순 그래프 기준



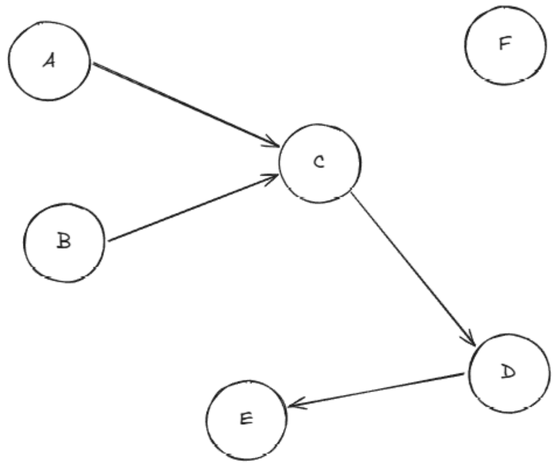
차수(Degree)의 분리

- 방향 그래프에서의 차수
  - 진입차수 (In-degree): 해당 정점으로 들어오는 화살표의 수
  - 진출차수 (Out-degree): 해당 정점에서 나가는 화살표의 수
  - B의 진입 차수는 2, 진출 차수는 2



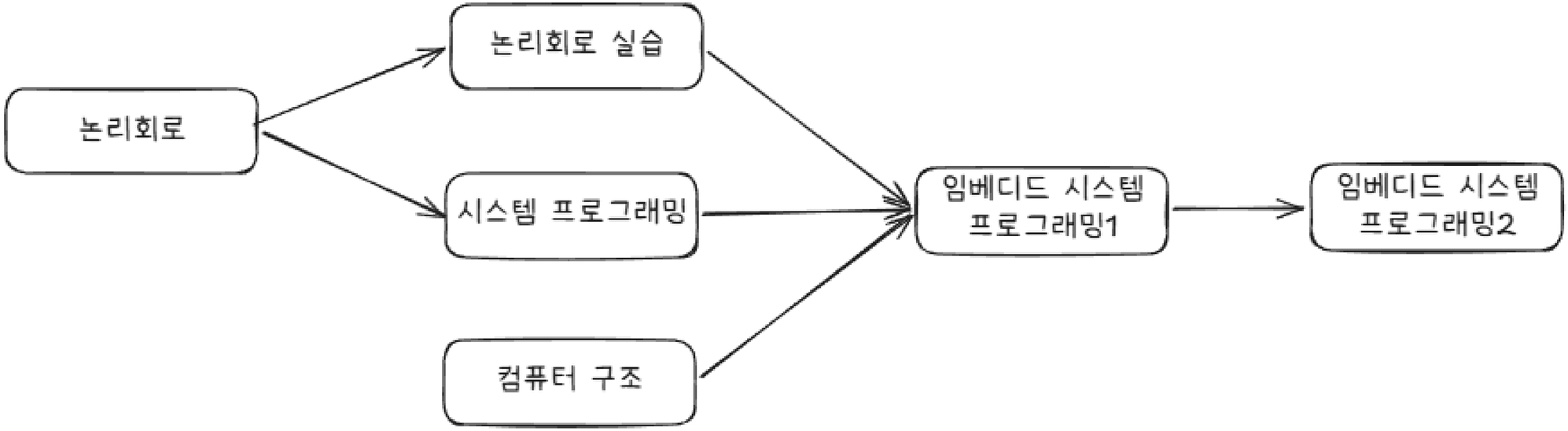
방향성 비순환 그래프 (DAG)

- DAG의 정의
  - 방향성(Directd): 모든 간선에는 특정 방향이 존재합니다.
  - 비순환(Acyclic): 순환(Cycle) 구조가 없습니다.
  - 그래프(Graph): 정점과 간선으로 구성된 자료구조입니다.



DAG의 예시

- 대학교 교과과정의 선수과목

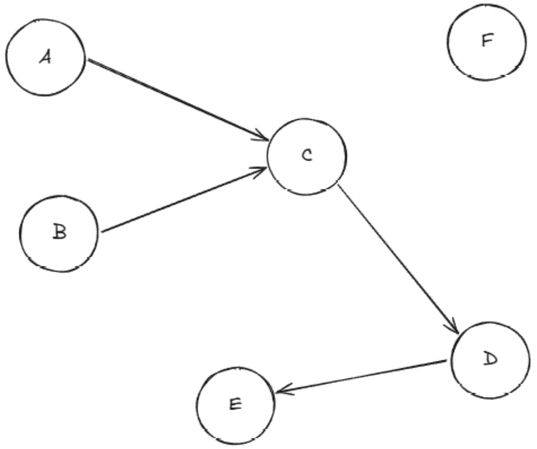


위상 정렬 (Topological Sort)

- 위상 정렬의 정의
  - DAG에 있는 모든 정점들을 방향성(선후 관계)에 어긋나지 않게 일렬로 나열하는 알고리즘입니다.
  - 복잡하게 얽힌 그래프를 순서대로 실행할 수 있는 하나의 타임라인으로 쫙 펴주는 역할을 합니다.
- 필수 전제 조건
  - 그래프가 사이클이 없는 방향 그래프(DAG) 여야 합니다.
  - 만약 사이클이 있다면, A를 하려면 B를 해야 하고, B를 하려면 A를 해야 하는 교착 상태(Deadlock)에 빠져 순서를 정할 수 없습니다.

위상 정렬 (Topological Sort)

- 다중 정답의 가능성
  - 위상 정렬의 결과는 하나의 정답만 존재하지 않습니다.
  - 선후 관계만 지킨다면, 여러 개의 올바른 위상 정렬 결과가 나올 수 있습니다.



위상 정렬 결과

A -> B -> C -> D -> E -> F

B -> A -> F -> C -> D -> E

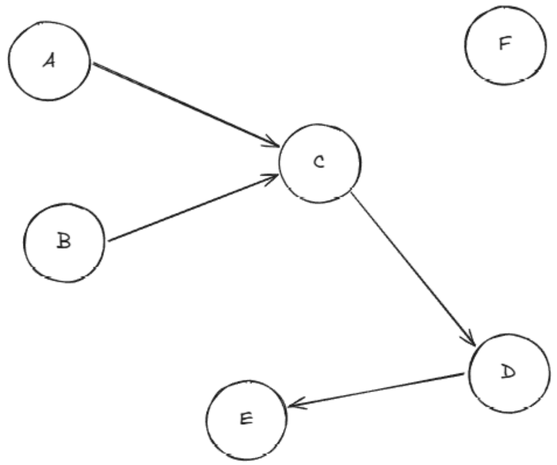
.
.
.

A -> B -> C -> F -> D -> E

Kahn의 알고리즘

- 핵심 동작 원리

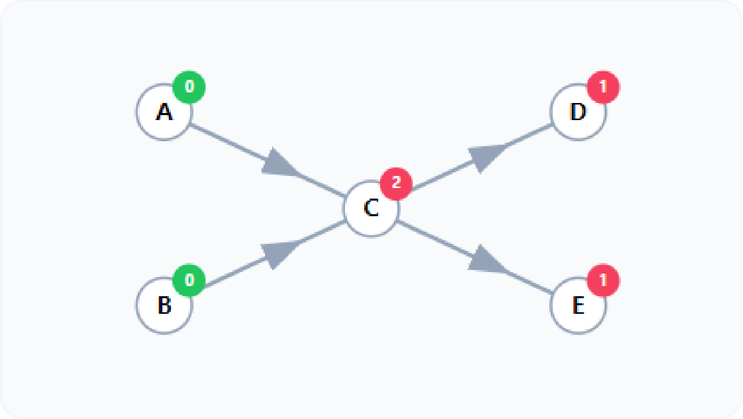
1. 지금 당장 할 수 있는 작업(진입차수 = 0)부터 처리합니다.
2. 작업이 끝났으면 해당 작업과 관련된 다음 작업들의 간선을 지웁니다.
3. 새롭게 진입차수가 0이 되는 작업이 없을 때까지 반복합니다.



Kahn의 알고리즘 시뮬레이션

큐(Queue)와 진입차수(In-degree)의 변화를 관찰해 보세요.

방향성 비순환 그래프 (DAG)



진입차수 0 (실행 가능) 진입차수 1 이상 (실행 불가) 작업 처리 중

초기 상태

각 노드의 진입차수(들어오는 화살표 개수)를 기록합니다. 아직 큐에는 아무것도 없습니다.

진입차수 (IN-DEGREE) 현황

A	B	C	D	E
0	0	2	1	1

대기열 큐 (QUEUE)

큐가 비어있습니다

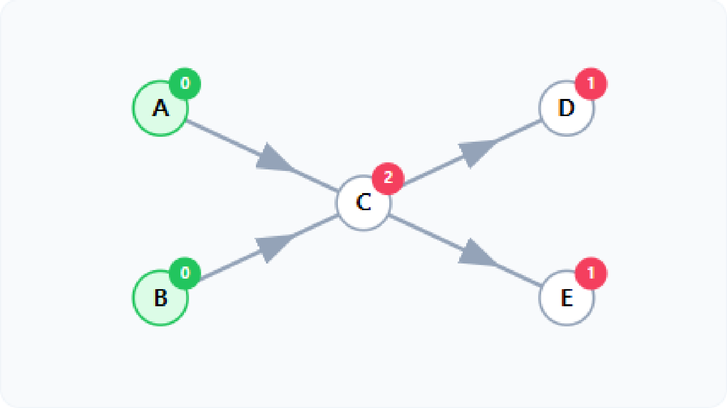
위상 정렬 결과 (RESULT)

아직 완료된 작업이 없습니다.

Kahn의 알고리즘 시뮬레이션

큐(Queue)와 진입차수(In-degree)의 변화를 관찰해 보세요.

방향성 비순환 그래프 (DAG)



진입차수 0 (실행 가능) 진입차수 1 이상 (실행 불가) 작업 처리 중

Step 1: 시작점 찾기 (Queue 삽입)

진입차수가 0인 노드(A, B)를 모두 찾아 큐(Queue)에 넣습니다. 이들이 바로 첫 번째 작업들입니다.

진입차수 (IN-DEGREE) 현황

A	B	C	D	E
0	0	2	1	1

대기열 큐 (QUEUE)

A B

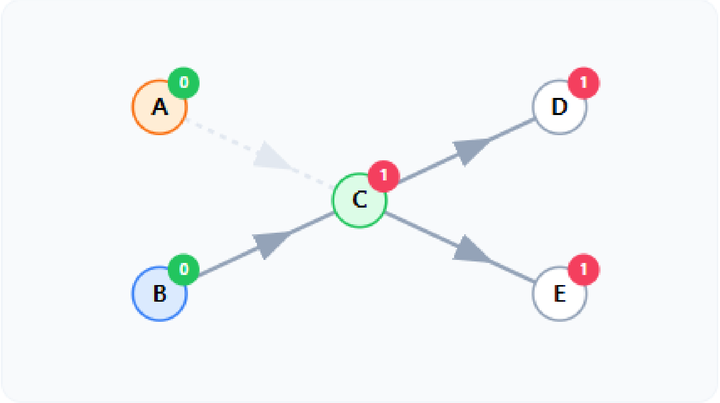
위상 정렬 결과 (RESULT)

아직 완료된 작업이 없습니다.

Kahn의 알고리즘 시뮬레이션

큐(Queue)와 진입차수(In-degree)의 변화를 관찰해 보세요.

방향성 비순환 그래프 (DAG)



진입차수 0 (실행 가능) 진입차수 1 이상 (실행 불가) 작업 처리 중

Step 2: 작업 A 처리 및 선행 조건 해제

큐에서 A를 꺼내 결과에 넣습니다. A가 끝났으므로, A에서 뻗어나가는 간선(A->C)을 제거하고 C의 진입차수를 1 깎아줍니다.

진입차수 (IN-DEGREE) 현황

A	B	C	D	E
0	0	1	1	1

대기열 큐 (QUEUE)

B

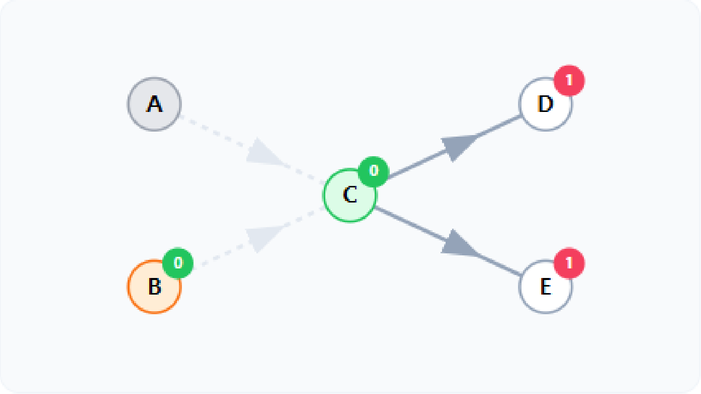
위상 정렬 결과 (RESULT)

A

Kahn의 알고리즘 시뮬레이션

큐(Queue)와 진입차수(In-degree)의 변화를 관찰해 보세요.

방향성 비순환 그래프 (DAG)



진입차수 0 (실행 가능) 진입차수 1 이상 (실행 불가) 작업 처리 중

Step 3: 작업 B 처리 (새로운 노드 발견!)

큐에서 B를 꺼냅니다. B->C 간선을 제거하니 C의 진입차수가 0이 되었습니다! 즉시 C를 큐에 밀어 넣습니다.

진입차수 (IN-DEGREE) 현황

A	B	C	D	E
0	0	0	1	1

대기열 큐 (QUEUE)

C

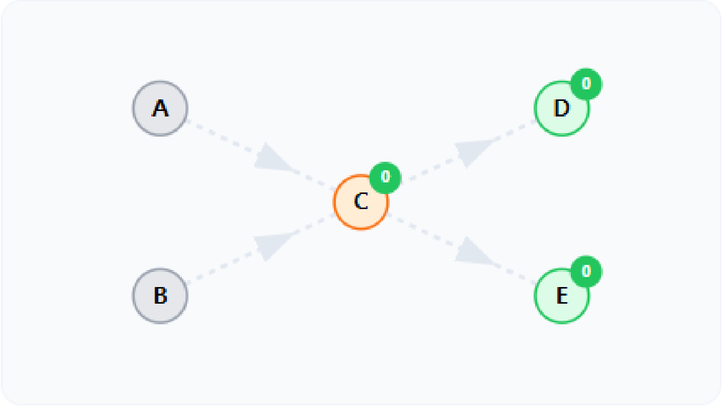
위상 정렬 결과 (RESULT)

A → B

Kahn의 알고리즘 시뮬레이션

큐(Queue)와 진입차수(In-degree)의 변화를 관찰해 보세요.

방향성 비순환 그래프 (DAG)



진입차수 0 (실행 가능) 진입차수 1 이상 (실행 불가) 작업 처리 중

Step 4: 작업 C 처리

큐에서 C를 꺼냅니다. C->D, C->E 간선을 제거하니 D와 E 모두 선행 조건이 해결(진입차수 0)되어 큐에 들어갑니다.

진입차수 (IN-DEGREE) 현황

A	B	C	D	E
0	0	0	0	0

대기열 큐 (QUEUE)

D E

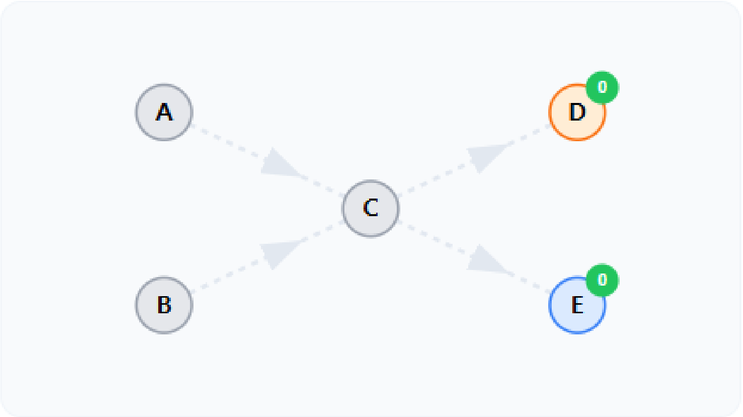
위상 정렬 결과 (RESULT)

A → B → C

Kahn의 알고리즘 시뮬레이션

큐(Queue)와 진입차수(In-degree)의 변화를 관찰해 보세요.

방향성 비순환 그래프 (DAG)



진입차수 0 (실행 가능) 진입차수 1 이상 (실행 불가) 작업 처리 중

Step 5: 작업 D 처리

큐에서 D를 꺼냅니다. D에서 나가는 화살표가 없으므로 다른 노드의 진입차수는 변하지 않습니다.

진입차수 (IN-DEGREE) 현황

A	B	C	D	E
0	0	0	0	0

대기열 큐 (QUEUE)

E

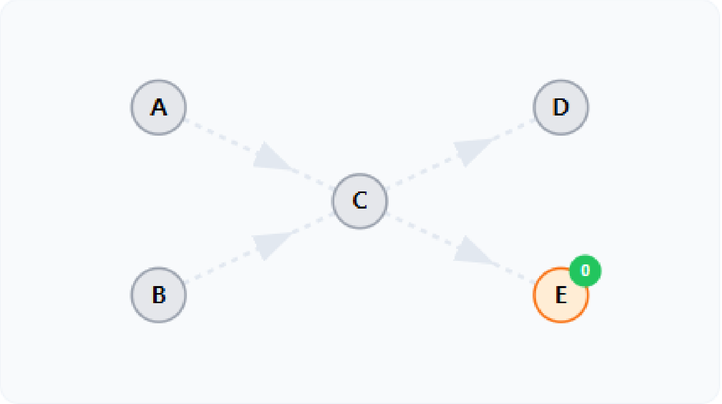
위상 정렬 결과 (RESULT)

A → B → C → D

Kahn의 알고리즘 시뮬레이션

큐(Queue)와 진입차수(In-degree)의 변화를 관찰해 보세요.

방향성 비순환 그래프 (DAG)



진입차수 0 (실행 가능) 진입차수 1 이상 (실행 불가) 작업 처리 중

Step 6: 작업 E 처리 및 완료

마지막으로 E를 꺼내어 결과에 추가합니다. 큐가 완전히 비었으므로 위상 정렬이 성공적으로 완료되었습니다!

진입차수 (IN-DEGREE) 현황

A	B	C	D	E
0	0	0	0	0

대기열 큐 (QUEUE)

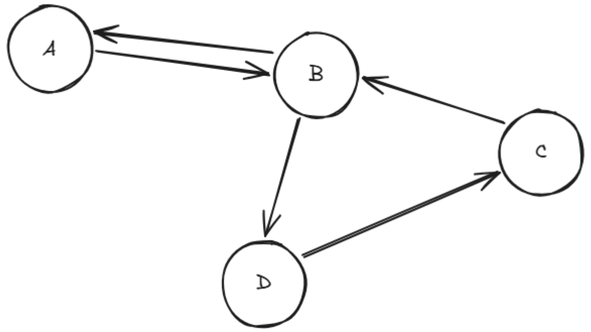
큐가 비어있습니다

위상 정렬 결과 (RESULT)

A → B → C → D → E

사이클 판별

- Kahn의 알고리즘 부가 기능
  - Kahn의 알고리즘은 단순히 순서를 세우는 것을 넘어서, 그래프 내에 사이클이 있는지 찾아낼 수 있습니다.
  - 모든 정점(전체 V 개)을 다 방문하기도 전에 큐(Queue)가 텅 비어버린다면, 사이클이 존재한다고 판정할 수 있습니다.
  - A, B 둘 다 화살표를 지워주기만을 기다리며 진입차수가 영원히 0이 되지 않습니다.



Kahn의 알고리즘 구현

```
// Kahn의 알고리즘을 사용하여 방향성 비순환 그래프(DAG)를 위상 정렬한다.
vector<int> topologicalSort(int N, const vector<vector<int>>& adj, vector<int>& in_degree) {
    queue<int> q;
    vector<int> result;
    // 진입 차수가 0인 정점을 큐에 삽입
    for (int i = 1; i <= N; ++i) {
        if (in_degree[i] == 0) {
            q.push(i);
        }
    }

    while (!q.empty()) {
        int curr = q.front();
        q.pop();
        result.push_back(curr);

        for (int next_node : adj[curr]) {
            in_degree[next_node]--;

            if (in_degree[next_node] == 0) {
                q.push(next_node);
            }
        }
    }

    // 큐가 빈 시점에 결과 배열에 담긴 정점의 수가 전체 정점의 수(N)보다 적다면 사이클이 존재함을 의미
    if (result.size() != N) {
        return {};
    }

    return result;
}
```

Kahn의 알고리즘 복잡도 분석

- 시간 복잡도 O(V + E)
  - 정점 방문 O(V): 큐(Queue)에 한 번 들어간 정점은 딱 한 번만 꺼내집니다.
    ※ 전체 정점을 한 번씩만 처리
  - 간선 확인 O(E): 큐에서 정점을 꺼낸 후, 그 정점에 연결된 간선들을 타고 가며 진입차수를 감소시킵니다.
    ※ 전체 간선은 정확히 한 번씩만 제거
- 공간 복잡도 O(V + E)
  - 인접 리스트 구조 O(V + E): 인접 리스트로 그래프를 구현할 경우, 정점과 간선 개수만큼의 메모리만 사용합니다.
  - 추가 배열과 큐 O(V): 진입차수를 기록하는 in_degree 배열, 대기열인 큐(Queue), 결과를 담는 result 배열 모두 최대 V개의 공간만 차지합니다.

줄 세우기 BOJ 2252

- 문제
  - 1번부터 N번까지의 학생들을 키 순서대로 줄을 세워야 합니다.
  - 두 학생의 키를 비교한 결과(A가 B 앞에 서야 함)가 M번 주어집니다.
  - 주어진 비교 결과를 바탕으로 모든 학생을 줄 세운 결과를 출력하며, 답이 여러 가지인 경우 아무거나 출력해도 됩니다.

줄 세우기 BOJ 2252

- 문제
  - 1번부터 N번까지의 학생들을 키 순서대로 줄을 세워야 합니다.
  - 두 학생의 키를 비교한 결과(A가 B 앞에 서야 함)가 M번 주어집니다.
  - 주어진 비교 결과를 바탕으로 모든 학생을 줄 세운 결과를 출력하며, 답이 여러 가지인 경우 아무거나 출력해도 됩니다.
- 해설
  - 명확한 선후 관계(방향성)가 주어지고 순서를 결정해야 하므로 위상 정렬을 사용해야 합니다.

    ※ A가 B 앞에 서야 하므로 A -> B 형태의 방향 그래프로 모델링
  - 진입차수(In-degree) 배열을 만들어, 자신보다 먼저 서야 하는(키가 작은) 학생의 수를 기록합니다.

    ※ 비교 결과가 입력될 때마다 뒤에 서는 학생의 진입차수를 1 증가
  - 진입차수가 0인 학생을 큐에 넣고, 큐에서 꺼낼 때마다 연결된 다음 학생들의 진입차수를 1씩 감소시킵니다.

    ※ 결과를 별도의 배열에 담지 않고, 큐에서 학생을 꺼낼 때(pop) 즉시 출력하면 메모리를 최적화

줄 세우기 BOJ 2252

```
int N, M;
cin >> N >> M;
vector<vector<int>> adj(N + 1);
vector<int> in_degree(N + 1, 0);

for (int i = 0; i < M; ++i) {
    int A, B;
    cin >> A >> B;
    adj[A].push_back(B);
    in_degree[B]++;
}

queue<int> q;
for (int i = 1; i <= N; ++i) {
    if (in_degree[i] == 0) {
        q.push(i);
    }
}

// 결과를 별도 배열에 저장하지 않고 즉시 출력하여 메모리 할당 오버헤드를 방지
while (!q.empty()) {
    int curr = q.front();
    q.pop();

    cout << curr << " ";

    for (int next_node : adj[curr]) {
        in_degree[next_node]--;
        if (in_degree[next_node] == 0) {
            q.push(next_node);
        }
    }
}
return 0;
```

DP와 DAG

- 모든 DP(동적 계획법) 문제는 DAG 구조를 이룬다.
  - DP의 상태(State) → 그래프의 노드(Node)
  - DP의 점화식 → 노드를 잇는 방향 간선(Directed Edge)
  - DP의 기초 값(Base Case) → 진입차수가 0인 시작 노드
- 사이클(Cycle)이 없어야 하는 이유
  - A를 구하기 위해 B가, B를 위해 C가, C를 위해 다시 A가 필요한 상황(사이클)이 발생하면 영원히 값을 확정할 수 없습니다.

    ※ 선후 관계가 명확하지 않으면 DP의 핵심인 메모이제이션(기록) 자체가 불가능
- 통찰
  - DP 문제는 보이지 않을 뿐, 내부적으로는 과거에서 미래로만 흐르는 일방통행 그래프(DAG) 형태를 띠고 있습니다.

작업 BOJ 2056

- 문제
  - 1번부터 N번까지의 작업이 있으며, 각 작업을 완료하는 데 걸리는 시간이 주어집니다.
  - 일부 작업은 반드시 먼저 완료되어야 하는 선행 작업이 존재합니다.

    ※ 선행 작업들의 번호는 항상 현재 작업의 번호보다 작음
  - 주어진 관계를 바탕으로 모든 작업을 완료하기 위한 최소 시간을 출력해야 합니다.

작업 BOJ 2056

- 문제
  - 1번부터 N번까지의 작업이 있으며, 각 작업을 완료하는 데 걸리는 시간이 주어집니다.
  - 일부 작업은 반드시 먼저 완료되어야 하는 선행 작업이 존재합니다.

    ※ 선행 작업들의 번호는 항상 현재 작업의 번호보다 작음
  - 주어진 관계를 바탕으로 모든 작업을 완료하기 위한 최소 시간을 출력해야 합니다.
- 해설
  - 현재 작업을 완료하는데 걸리는 최단시간은 선행 작업에 걸린 총 시간 + 자신의 작업 시간입니다.
  - 1번부터 N번 작업까지 순회하며, dp[다음 작업] = max(기존 값, 내 완료 시간 + 다음 작업 시간)으로 갱신합니다

    ※ 선행 작업의 번호가 항상 더 작다는 조건 덕분에, 번호순으로 탐색하는 것 자체가 위상 정렬이 되어 복잡한 큐(Queue) 구현을 생략할 수 있음

작업 BOJ 2056

```
vector<int> time_cost(N + 1);
vector<vector<int>> adj(N + 1);
vector<int> dp(N + 1);

for (int i = 1; i <= N; ++i) {
    int pre_count;
    cin >> time_cost[i] >> pre_count;

    // 선행 조건 유무와 관계없이, 모든 작업의 기본 소요 시간을 자기 자신으로 세팅
    dp[i] = time_cost[i];

    for (int j = 0; j < pre_count; ++j) {
        int pre;
        cin >> pre;
        adj[pre].push_back(i);
    }
}

for (int curr = 1; curr <= N; ++curr) {
    for (int next_node : adj[curr]) {
        // 다음 작업(next_node)으로 진입하는 여러 경로 중, 가장 늦게 끝나는 선행 작업의 시간을 선택
        dp[next_node] = max(dp[next_node], dp[curr] + time_cost[next_node]);
    }
}

return *max_element(dp.begin() + 1, dp.end());
```

그 외 연습 문제

- 14567 선수과목 (Prerequisite)
- 2623 음악프로그램
- 1005 ACM Craft
- 5021 왕위 계승